B. Buying Lemonade

time limit per test 1 second
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1100 | Tags: binary search, constructive algorithms, sortings | https://codeforces.com/problemset/problem/2024/B

There is a vending machine that sells lemonade. The machine has a total of n slots. You know that initially, the i-th slot contains a_i cans of lemonade. There are also n buttons on the machine, each button corresponds to a slot, with exactly one button corresponding to each slot. Unfortunately, the labels on the buttons have worn off, so you **do not know** which button corresponds to which slot.

When you press the button corresponding to the i-th slot, one of two events occurs:

- If there is a can of lemonade in the i-th slot, it will drop out and you will take it. At this point, the number of cans in the i-th slot decreases by 1.
- If there are no cans of lemonade left in the i-th slot, nothing will drop out.

After pressing, the can drops out so quickly that it is impossible to track from which slot it fell. The contents of the slots are hidden from your view, so you cannot see how many cans are left in each slot. The only thing you know is the initial number of cans in the slots: $a_1, a_2, \ldots, a_n$.

Determine the minimum number of button presses needed to guarantee that you receive at least k cans of lemonade.

Note that you can adapt your strategy during the button presses based on whether you received a can or not. It is guaranteed that there are at least k cans of lemonade in total in the machine. In other words, $k \le a_1 + a_2 + \ldots + a_n$.

Input

Each test consists of multiple test cases. The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains two integers n and k ($1 \le n \le 2 \cdot 10^5$, $1 \le k \le 10^9$) — the number of slots in the machine and the required number of cans of lemonade.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the number of cans in the slots.

It is guaranteed that $k \le a_1 + a_2 + \ldots + a_n$, meaning there are at least k cans of lemonade in the machine.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the minimum number of button presses needed to guarantee that you receive at least k cans of lemonade.

Example

Input

```
5
2 1
1 1
2 2
1 2
3 4
2 1 3
10 50
1 1 3 8 8 9 12 13 27 27
2 1000000000
1000000000 500000000
```

Output

```
1
2
5
53
1000000000
```

Note

In the first test case, we can simply press the first button and receive one can of lemonade.

In the second test case, we can press each button once and guarantee that we receive 2 cans of lemonade. Note that if we simply press one button twice, we might not be lucky, and that button could correspond to the first slot, in which case we would only receive 1 can of lemonade for two presses.

In the third test case, one of the optimal strategies is as follows:

Press the first button twice. After the first press, a can of lemonade will definitely drop out. Then there are two options:

- If no can of lemonade drops after the second press, we know that this button must correspond to the second slot, since $a_2 = 1$ and a_1, a_3 gt 1. Then we can press the second button twice and the third button once. Since $a_1, a_3 \geq 2$, we will definitely receive three cans of lemonade for these three presses. Thus, after 5 presses, we will have 4 cans of lemonade.
- If a can of lemonade drops after the second press, we can make one press on the second button and one press on the third button. After each of these presses, we will definitely receive a can of lemonade. Thus, after 4 presses, we will have 4 cans of lemonade.

It can be shown that it is impossible to guarantee receiving 4 cans of lemonade with only 4 presses, so the answer is 5.